

# Documentación de trabajo — Proyecto WiFiConnect

**Fecha:** 29 de julio de 2026

**Proyecto:** WiFiConnect

**Ruta local:** `C:\Users\aleja\OneDrive\Documentos\APP`

---

## 1. Objetivo del proyecto

WiFiConnect es una aplicación web para gestionar servicios de internet. La arquitectura está dividida en:

- **Frontend:** React + Vite + Tailwind CSS.
- **Backend:** Node.js + Express.
- **Base de datos:** SQL Server.
- **Autenticación:** JWT + bcryptjs.
- **Cliente HTTP:** Axios.

El sistema contempla usuarios públicos, clientes y administradores. El panel de administrador incluye gestión de clientes, planes, zonas, tickets, noticias y reportes. El panel de cliente incluye servicio, facturas, tickets y perfil.

---

## 2. Estado inicial que encontramos

El proyecto originalmente estaba preparado para **MySQL**. Había referencias a `mysql2`, `mysql2/promise`, `DB_PORT=3306`, `root` y consultas con `?`.

Después decidimos trabajar con **SQL Server**, por lo que se inició la migración del backend y de los scripts SQL.

En el proyecto existen las carpetas principales:

```
APP/  
├─ backend/  
├─ frontend/  
└─ database/
```

La estructura documentada incluye backend con configuración, controladores, middleware, rutas, servicios y utilidades; frontend con React, layouts, páginas y componentes; y scripts SQL para la base de datos.

---

### 3. SQL Server utilizado

En SQL Server Management Studio (SSMS) comprobamos que el servidor funciona y que la conexión utilizada es:

```
Servidor:      LUISVILLA\MSSQLSERVER01
Autenticación: Autenticación de Windows
Usuario:       LUISVILLA\aleja
```

También comprobamos desde PowerShell que existen dos servicios SQL Server activos:

```
MSSQLSERVER
MSSQL$MSSQLSERVER01
```

La base utilizada por el proyecto es:

```
wifi_connect
```

La base ya existía, por lo que al ejecutar `CREATE DATABASE wifi_connect;` SQL Server respondió que la base ya estaba creada. Eso confirmó que no había que volver a crearla.

---

### 4. Base de datos y estructura

El proyecto WiFiConnect maneja las siguientes entidades principales:

```
usuarios
planes
zonas
cobertura
clientes
facturas
tickets
noticias
notificaciones
faq
```

Durante la migración a SQL Server se adaptaron conceptos de MySQL como:

```
AUTO_INCREMENT → IDENTITY(1,1)
TINYINT(1)      → BIT
```

```
ENUM          → VARCHAR + CHECK
mysql2/promise → mssql
```

La estructura SQL Server documentada usa `IDENTITY`, `BIT`, `CHECK`, `DATETIME`, `GETDATE()` y claves foráneas.

---

## 5. Datos iniciales

Se prepararon datos de prueba para:

- Un administrador.
- Un cliente.
- Cinco planes de internet.
- Cinco zonas de cobertura.
- Preguntas frecuentes.
- Noticias y avisos.
- Una relación de cliente con plan y zona.

El script de datos contiene el usuario administrador con correo:

```
admin@wificonnect.com
```

y un hash bcrypt para la contraseña. También existe un usuario cliente con:

```
cliente@wificonnect.com
```

Importante: se detectó que algunas versiones antiguas del archivo contenían el marcador `PEGAR_AQUI_HASH_ADMIN`, mientras que otra versión contiene un hash real. Por eso todavía se está verificando cuál contraseña corresponde realmente al hash almacenado.

---

## 6. Migración del backend de MySQL a SQL Server

Se eliminó `mysql2` del backend:

```
npm uninstall mysql2
```

Después se instaló:

```
npm install mssql
npm install msnodesqlv8
```

El `package.json` del backend quedó con `mssql` y `msnodesqlv8`, sin `mysql2`.

También se añadió:

```
"type": "module"
```

para que Node.js interprete correctamente los archivos que utilizan `import` y `export`.

---

## 7. Configuración actual de `db.js`

El archivo correcto para la conexión SQL Server es:

```
backend/src/config/db.js
```

Se reemplazó la conexión antigua de MySQL por una conexión usando:

```
import sql from 'mssql/msnodesqlv8.js';
import dotenv from 'dotenv';
```

La conexión usa `trustedConnection: true`, porque SQL Server se está utilizando con autenticación de Windows.

La configuración también contiene una función `query()` para ejecutar consultas y convertir parámetros `?` a parámetros SQL Server.

---

## 8. Variables `.env`

La intención de configuración actual es:

```
PORT=4000
NODE_ENV=development

DB_HOST=LUISVILLA\MSSQLSERVER01
DB_NAME=wifi_connect
DB_TRUSTED_CONNECTION=true
```

```
JWT_SECRET=clave_super_secreta_wifi_connect_2026
JWT_EXPIRES_IN=2h

MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USER=
MAIL_PASS=
MAIL_FROM=WiFiConnect <noreply@wificonnect.com>

RATE_LIMIT_WINDOW=15
RATE_LIMIT_MAX=100
```

No se está usando `sa` ni una contraseña SQL en esta configuración, porque la conexión de SSMS utiliza Windows Authentication.

El correo de Nodemailer quedó temporalmente sin configurar porque es opcional para esta etapa.

---

## 9. Frontend y separación de responsabilidades

Se detectó un error importante: se había colocado código de `mssql` y `dotenv` dentro de:

```
frontend/src/services/api.js
```

Eso es incorrecto.

El frontend debe comunicarse con el backend mediante Axios y **nunca debe conectarse directamente a SQL Server**.

La arquitectura correcta es:

```
React / Vite
  ↓
Axios
  ↓
Node + Express
  ↓
mssql / msnodesqlv8
  ↓
SQL Server
  ↓
wifi_connect
```

Por eso `frontend/src/services/api.js` debe contener solamente la configuración de Axios y el interceptor JWT.

---

## 10. Sistema de inicio de sesión

El apartado de login ya está implementado en el frontend.

La página permite introducir:

Correo  
Contraseña

El frontend llama a:

POST /api/auth/login

El backend recibe `correo` y `contraseña`, busca el usuario activo y compara la contraseña con `bcrypt.compare()`.

Cuando la contraseña no coincide, devuelve:

Contraseña incorrecta

Por eso el mensaje mostrado en la pantalla confirma que el formulario y la ruta de login están llegando hasta la lógica de autenticación.

---

## 11. Credenciales de aplicación

Se ha definido como usuario administrador:

Correo: admin@wificonnect.com

También existe un usuario cliente de prueba:

Correo: cliente@wificonnect.com

Las contraseñas de prueba que aparecieron durante la documentación fueron `Admin123` y `Cliente123`, pero **no deben considerarse válidas hasta confirmar que el hash almacenado en SQL Server corresponde a esas contraseñas.**

Para establecer una contraseña conocida se propuso generar un hash con `bcryptjs` desde el backend y actualizar el registro en SQL Server.

---

## 12. Problemas encontrados y solucionados

### Problema 1 — `mysql2` no encontrado

Error:

```
Cannot find package 'mysql2'
```

Causa: `mysql2` fue desinstalado, pero `db.js` todavía lo importaba.

Solución: reemplazar la conexión MySQL por SQL Server.

---

### Problema 2 — import de `mssql/msnodesqlv8`

Error:

```
Cannot find module 'mssql/msnodesqlv8'  
Did you mean to import "mssql/msnodesqlv8.js"?
```

Solución:

```
import sql from 'mssql/msnodesqlv8.js';
```

---

### Problema 3 — warning de módulos ES

Node mostraba:

```
MODULE_TYPELESS_PACKAGE_JSON
```

Solución: agregar en `backend/package.json`:

```
"type": "module"
```

## Problema 4 — ODBC

El backend llegó correctamente hasta el intento de conexión, pero apareció:

```
[Microsoft][Administrador de controladores ODBC]  
No se encuentra el nombre del origen de datos y no se especifica ningún  
controlador predeterminado
```

Se comprobó desde PowerShell que **sí están instalados**:

```
SQL Server – 32-bit  
ODBC Driver 17 for SQL Server – 32-bit  
ODBC Driver 18 for SQL Server – 32-bit  
SQL Server – 64-bit  
ODBC Driver 17 for SQL Server – 64-bit  
ODBC Driver 18 for SQL Server – 64-bit
```

Por lo tanto, no es necesario instalar otro ODBC. El siguiente paso es verificar/configurar correctamente el driver utilizado por `msnodesqlv8`.

## 13. Problema actual del login

La interfaz de login funciona y muestra:

```
Contraseña incorrecta
```

El usuario utilizado es:

```
admin@wificonnect.com
```

Se encontró en los scripts una versión con un hash real:

```
$2a$10$3n2yEPbn6CCqUYE8yGVLa0kb/deevY8LhB6jGktLTtMsRrgYT33oW
```

Pero todavía no se ha comprobado qué contraseña corresponde exactamente a ese hash.



Para solucionar esto se propuso generar un hash nuevo para una contraseña conocida, por ejemplo:

Admin123

usando `bcryptjs`, y después actualizar el registro del administrador en SQL Server.

---

## 14. Comandos utilizados hoy

Desde `backend`:

```
npm uninstall mysql2
npm install mssql
npm install msnodesqlv8
npm run dev
```

Para comprobar drivers ODBC:

```
Get-OdbcDriver | Where-Object {$_.Name -like "**SQL Server*"} | Select-Object
Name, Platform, Version
```

---

## 15. Estructura objetivo del proyecto

```
APP/
|
|— backend/
|   |
|   |— src/
|   |   |
|   |   |— config/
|   |   |   |
|   |   |   |— db.js
|   |   |   |— mail.js
|   |   |— controllers/
|   |   |— middleware/
|   |   |— routes/
|   |   |— services/
|   |   |— repositories/
|   |   |— validators/
|   |   |— utils/
|   |   |— app.js
|   |   |— server.js
|   |— uploads/
```

```
|   |   | .env
|   |   | └─ package.json
|   |
|   └─ frontend/
|       |   |   | src/
|       |   |   |   |   | components/
|       |   |   |   |   | layouts/
|       |   |   |   |   | pages/
|       |   |   |   |   | context/
|       |   |   |   |   | services/
|       |   |   |   |   |   └─ api.js
|       |   |   |   |   └─ App.jsx
|       |   |   |   |   └─ main.jsx
|       |   |   |   └─ vite.config.js
|       |   |   └─ package.json
|       |
|       └─ database/
|           |   |   | wifi_connect.sql
|           |   |   | seed.sql
|           |   |   └─ backup/
```

## 16. Estado al finalizar la sesión

### ✓ Listo

- SQL Server instalado y funcionando.
- Instancia identificada: `LUISVILLA\MSSQLSERVER01`.
- Base `wifi_connect` existente.
- SSMS conectado mediante Windows Authentication.
- `mysql2` eliminado del backend.
- `mssql` instalado.
- `msnodesqlv8` instalado.
- Drivers ODBC 17 y 18 presentes en Windows.
- Frontend con React/Vite.
- API del frontend preparada para Axios.
- Login visual implementado.
- Ruta `/api/auth/login` implementada.
- JWT y bcryptjs presentes.

### 🕒 Pendiente

1. Terminar la conexión Node.js → SQL Server usando el driver ODBC correcto.
2. Verificar que el backend muestre `SQL Server conectado correctamente`.
3. Probar una consulta real desde Node hacia `wifi_connect`.
4. Confirmar que la tabla `usuarios` contiene el administrador correcto.

5. Generar/establecer un hash conocido para `Admin123` y probar el login.
  6. Corregir cualquier controlador que todavía conserve imports o consultas de la versión MySQL.
  7. Probar el resto de endpoints.
  8. Dejar Nodemailer configurado cuando se quiera activar el correo.
- 

## 17. Próximo paso recomendado

No modificar la base de datos ni volver a instalar SQL Server.

El próximo objetivo es conseguir primero:

- ✓ SQL Server conectado correctamente
- ✓ Base de datos: `wifi_connect`
- ✓ Servidor corriendo en `http://localhost:4000`

Después se prueba:

```
GET /  
POST /api/auth/login
```

y finalmente el acceso del administrador desde:

```
http://localhost:5173/login
```

**Nota:** esta documentación registra el estado real del trabajo y distingue lo que está terminado de lo que todavía necesita comprobación.